Day 8 2nd Dec 2025

→ While loops :-

It is a control statiment that repeatidly executes a block of code as long as a given condition is true

Syntax :-

```
while (condition):
    statements
    increment / dicrement
```

Ex:-
```
num = int (input("enter a number: "))
sum = 0
while (num > 0):
    rem = num % 10
    sum = sum + rem
    num = num // 10
print ("The sum of values:", sum)
```

output enter a number : 1234
The sum of value : 10

1 palindrome

* Chuck the given no is palindrome @ not
Before this what is palindrome

→ palindrome is a no @ word that reads
Ex:- 121, 1331, level, wow, madam etc...

-> chuck it's a palindrome or not.

```
=> num = int (input ("enter a no: "))
   temp = num
   rev = 0

   while num > 0:
         rem = num % 10
         rev = rev * 10 + rem
         num = num // 10

   print (temp, rev)

   if (temp == rev):
         print("its a palindrome")
   else:
         print ("its not a palindrome.
```

2. Fibonacci series

It is a series of squence where each number is the "sum of the previous two numbers"

```
Ex:- a = 0
     b = 1
     count = 0

     while count < n:
           print (a)
           next = a + b
           a = b
           b = next
           count += 1
```

Day 9 Dec 11th 2025

Functions

A function is a block of code that performs the specific task that helps in breaking a large program into smaller managable.

Why function is used.

1) To increase the code reusability
2) To improve the readability & modularity
3) To Reduce the file size
4) Make easier debugging & tuting

what are the types of function?

1) user friendly function
   → It is created by the programmer using "def"
2) Built in function
   → provided by python ln(), print() etc...
3) lambda function
   → Anonymous single line function created with lambda

1) User friendly function

Methods to creat a user friendly function

1) without input and without return value

Syntax

```
def fun_name():
    code block

call function
fun_name()
```

Ex:-
```
def evn_odd():
    num = int(input("enter a value: "))
    if (num % 2 == 0):
        print(num, "is evn")
    else:
        print(num, "is odd")
```

2) with input and without return value

Syntax

```
def fun_name(P1, P2 ---P4):
    code block

call function
fun_name(P1, P2 - - P4).
```

Ex:- def fun_name

```
def even_odd_1(n):
    if (n%2==0):
        print (n, "is even")
    else:
        print (n," is odd")

even_odd_1(5)
```

3) without input and with return value.

Syntax

```
def fun_name():
    Code block return value

call function
fun_name()
```

Ex:-

```
def even_odd_2():
    if (num = int(input("enter a value: "))
        if (num % 2 ==0):
            return (f"{num}, even")
        else:
            return(f"{num}, odd")

even_odd_2()
```

Ex:- ~~def fun_name~~

```
def even_odd_1(n):
    if (n%.2==0):
        print (n, "is even")
    else:
        print (n," is odd")

even_odd_1(5)
```

3) without input and with return value.

Syntax

```
def fun_name():
    Code block return value

call function
fun_name()
```

Ex:-

```
def even_odd_2():
    if (num = int (input (" enter a value:
        if (num %. 2 ==0):
            return (f" {num}, eve
        else:
            return (f" {num}, odd")

even_odd_2()
```

4) with input and with return value

Syntax

```
def fun_name(P1, P2 --- Pn):
    code block

    return value
call function
fun_name(P1, P2 --- Pn)
```

Ex:-

```
def even_odd_3(n1):
    value = int(input("enter a value: "))
    if (n1 % 2 == 0):
        return n1, "even"
    else:
        return n1, "odd"
print(even_odd_3(value))
```